

WhiteDavid23 Academy

Partner Portal / Admission CRM Platform

System Design Document — Version 1.0

Project: whitedavid23_academy

System Type: Multi-tenant Partner Portal / Admission CRM

Architecture: Modular Monolith

Primary Users: Super Admin, Partner Admin, Team Member/Counselor, Support

Document Status: Development-ready system design

Date: 2026-09-10

1. System Overview

whitedavid23_academy is a multi-tenant web-based CRM and admission management platform designed to connect WhiteDavid23 with partner academies, communities, counselors, and support personnel.

The platform manages the complete admission lifecycle:

```
Partner
  ↓
Lead Capture
  ↓
Lead Assignment
  ↓
Contact / Follow-up
  ↓
Demo / Brochure
  ↓
Fee Discussion
  ↓
Admission
  ↓
Payment
  ↓
Commission
  ↓
Reporting
```

The system must provide:

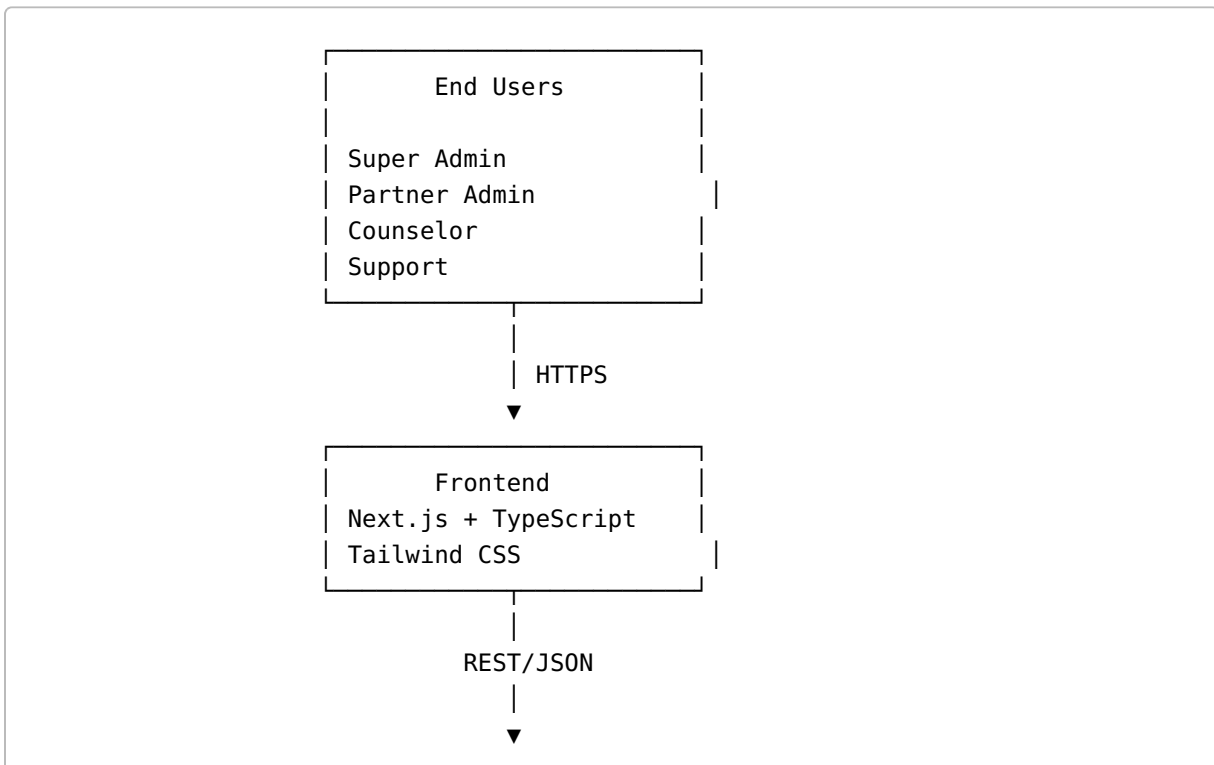
- strict partner-level data isolation
- role-based authorization
- lead lifecycle management
- follow-up management
- admission conversion
- course management
- commission calculation
- reporting and analytics
- notifications
- marketing assets
- audit logging
- import/export
- future integration capabilities

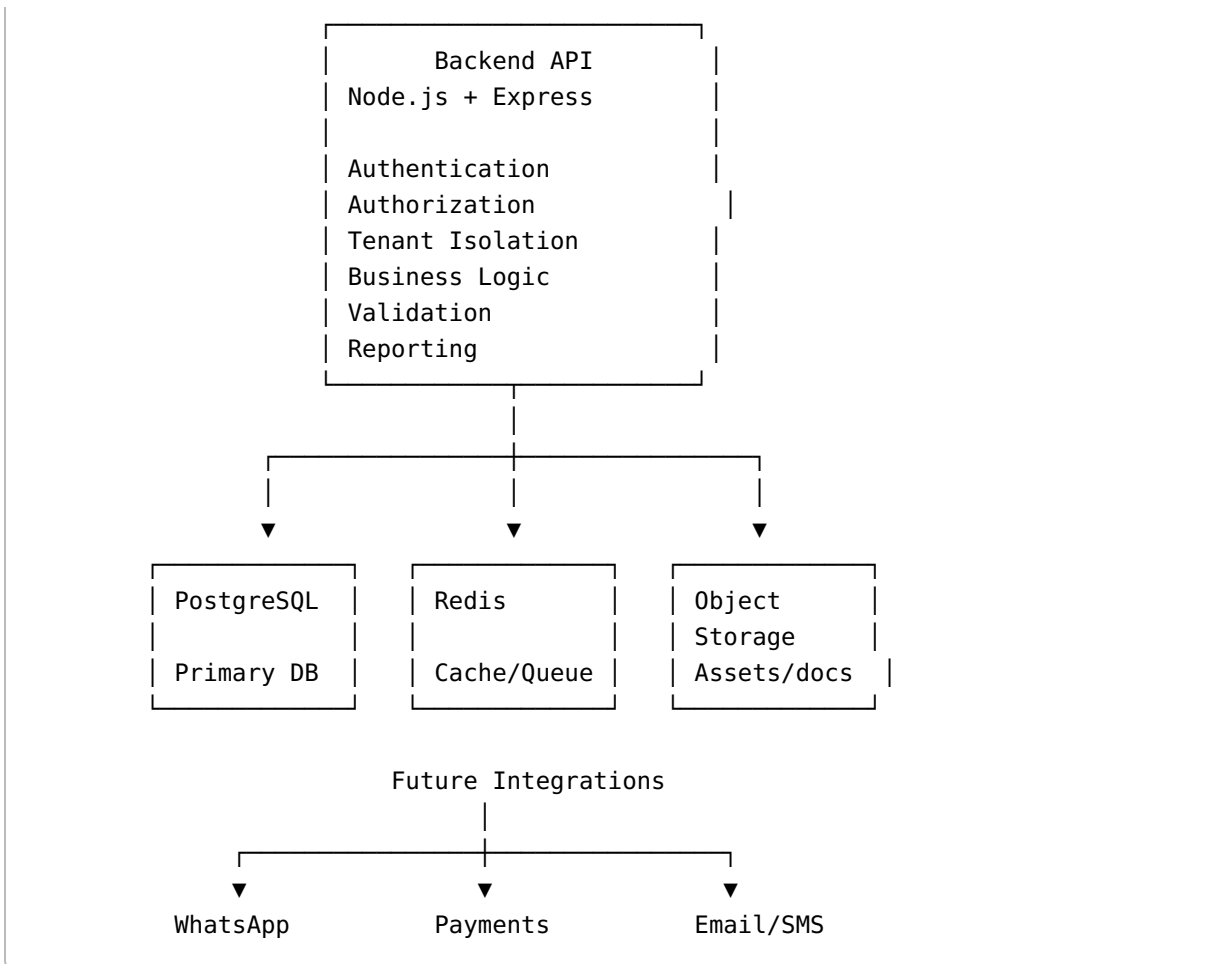
2. Recommended Architecture

2.1 Architectural Style

Recommended

Modular Monolith + REST API + PostgreSQL





3. Why Modular Monolith?

Microservices are **not recommended for Phase 1**.

The application has tightly related business domains:

```
Leads
  ↓
Follow-ups
  ↓
Admissions
  ↓
Payments
  ↓
Commissions
```

↓
Reports

Splitting these into independent services immediately would introduce:

- distributed transactions
- service discovery
- network failures
- additional deployment complexity
- duplicated authentication
- distributed logging
- eventual consistency problems

The project does not initially require that complexity.

Instead:

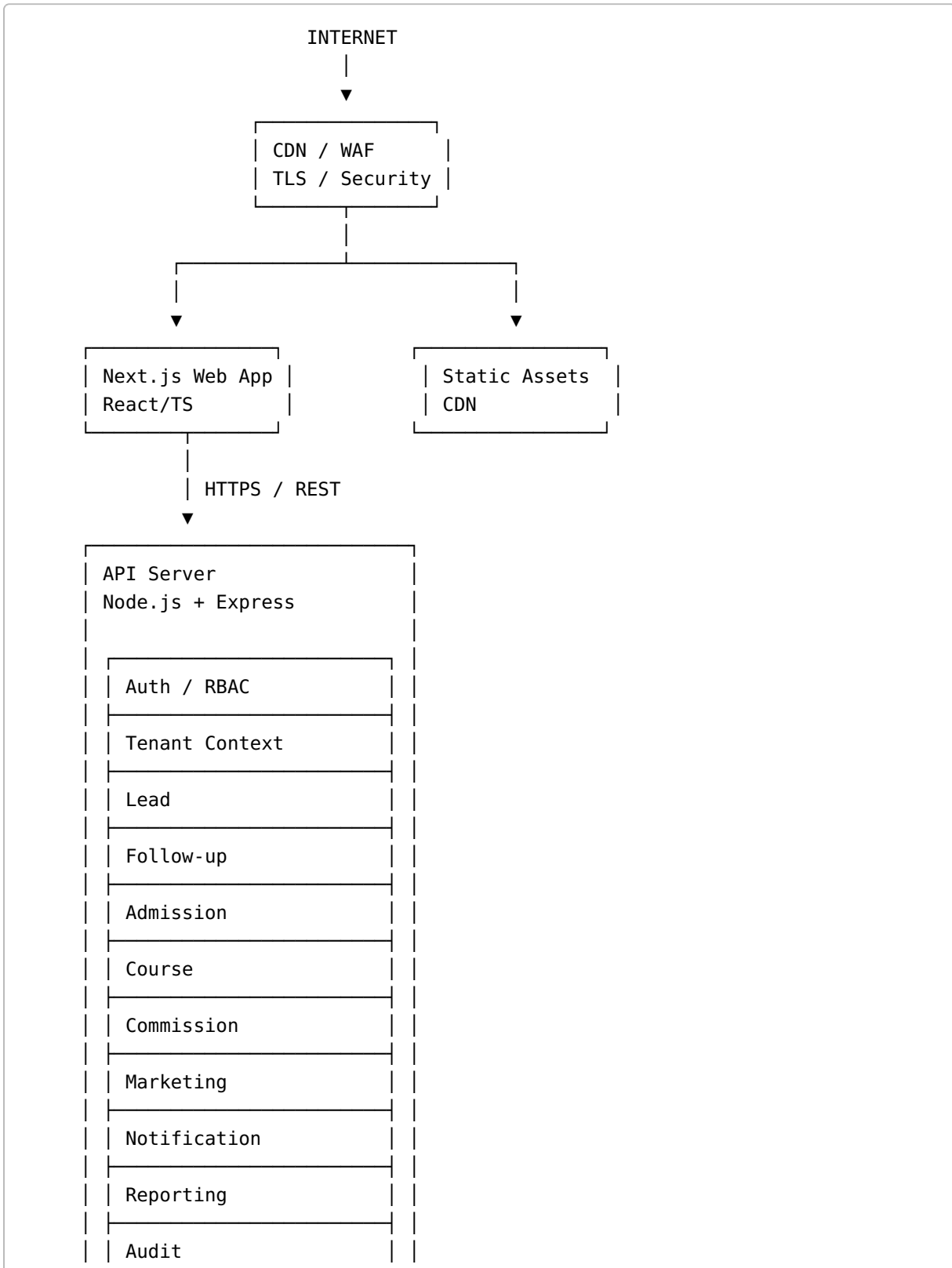
```
One Backend
|
├─ Auth Module
├─ User Module
├─ Partner Module
├─ Lead Module
├─ Follow-up Module
├─ Admission Module
├─ Course Module
├─ Marketing Module
├─ Commission Module
├─ Notification Module
├─ Reporting Module
├─ Audit Module
└─ Import/Export Module
```

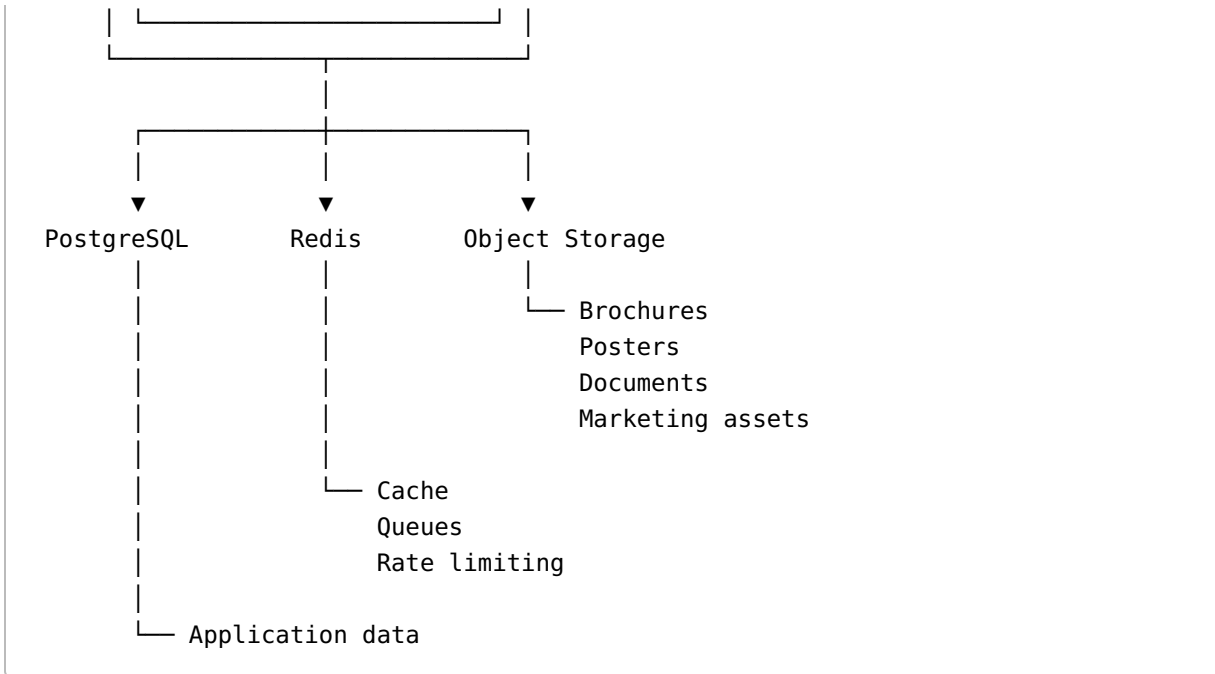
Each module has its own:

```
Controller
Service
Repository
Validation
DTO
Domain logic
```

This provides a clean migration path toward microservices later if scale actually demands it.

4. High-Level System Architecture



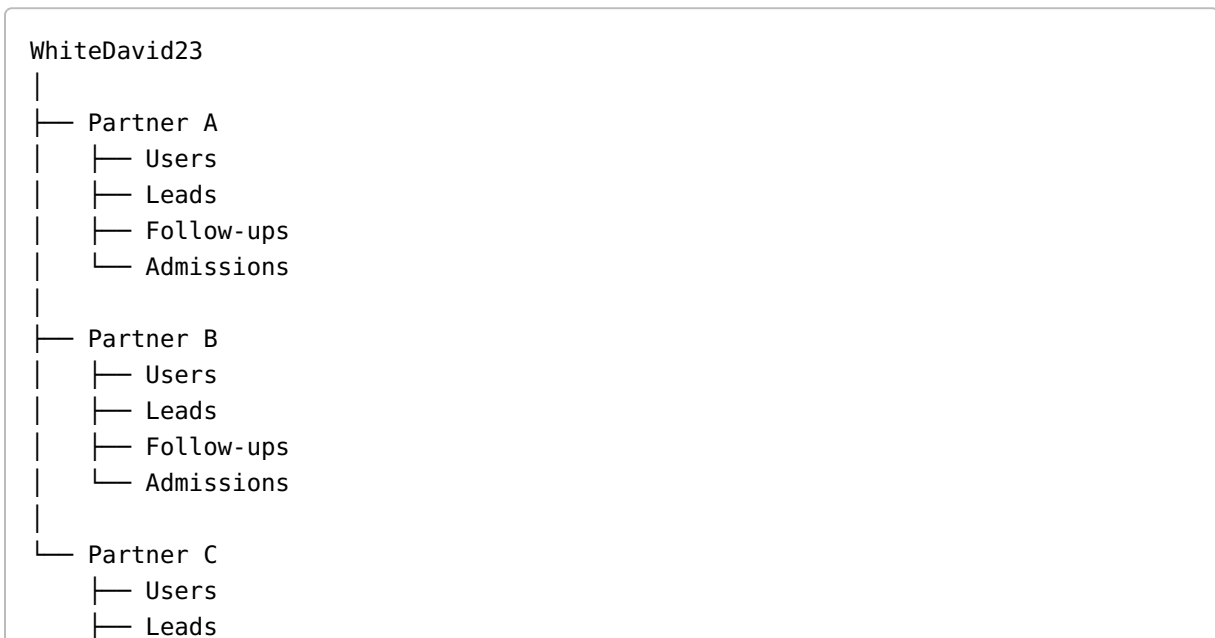


5. Multi-Tenant Architecture

This is one of the most important architectural requirements.

Each partner represents a tenant.

Example:



└─ Follow-ups
└─ Admissions

Tenant Identification

Most tenant-owned tables contain:

partner_id

Example:

```
leads
├──
id
partner_id
name
phone
email
...
```

A Partner Admin request:

`GET /api/leads`

must automatically become logically equivalent to:

```
SELECT *
FROM leads
WHERE partner_id = authenticated_user.partner_id;
```

The client must **not** be trusted to supply the partner ID.

Bad:

`GET /api/leads?partnerId=123`

Good:

JWT
↓

```
authenticated user
↓
user.partner_id
↓
tenant context
↓
database query
```

6. Tenant Isolation Rules

Super Admin

```
partner_id = NULL
```

or global administrative context.

Can access:

```
ALL PARTNERS
ALL LEADS
ALL ADMISSIONS
ALL REPORTS
ALL COMMISSIONS
```

Partner Admin

```
partner_id = authenticated user's partner_id
```

Can access:

```
ONLY THEIR PARTNER'S DATA
```

Team Member / Counselor

```
partner_id = authenticated user's partner_id
AND
assigned_to = authenticated user's id
```

Support

Access should be explicitly scoped to permitted support operations.

7. Core Domain Model

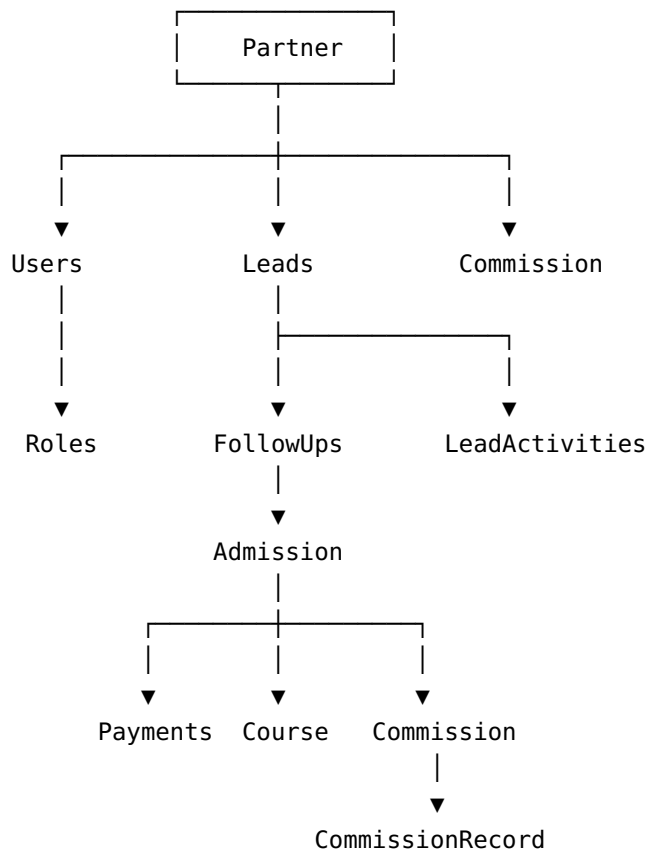
Primary entities:

```
User
Role
Permission
Partner
Lead
LeadActivity
FollowUp
Admission
Payment
Course
MarketingAsset
Campaign
CommissionRule
CommissionRecord
Notification
AuditLog
```

Potential supporting entities:

```
PartnerMember
LeadAssignment
AdmissionDocument
Announcement
ImportJob
ExportJob
RefreshToken
SystemSetting
```

8. Entity Relationship Model



9. Database Design

9.1 users

users

id	UUID PK
partner_id	UUID FK NULL
role_id	UUID FK
name	VARCHAR
email	VARCHAR UNIQUE
phone	VARCHAR
password_hash	TEXT
status	ENUM
last_login_at	TIMESTAMP

created_at	TIMESTAMP
updated_at	TIMESTAMP

10. partners

partners

id	UUID PK
academy_name	VARCHAR
partner_name	VARCHAR
owner_name	VARCHAR
email	VARCHAR
mobile	VARCHAR
address	TEXT
logo_url	TEXT
commission_type	VARCHAR
status	ENUM
joining_date	DATE
approved_at	TIMESTAMP
approved_by	UUID
created_at	TIMESTAMP
updated_at	TIMESTAMP

Statuses:

PENDING
ACTIVE
SUSPENDED
REJECTED

11. roles

roles

id
name
description

Default roles:

```
SUPER_ADMIN
PARTNER_ADMIN
COUNSELOR
SUPPORT
```

For fine-grained authorization, add:

```
permissions
```

```
id
code
description
```

Example permissions:

```
LEAD_VIEW
LEAD_CREATE
LEAD_UPDATE
LEAD_DELETE
LEAD_ASSIGN

ADMISSION_VIEW
ADMISSION_CREATE
ADMISSION_UPDATE

COMMISSION_VIEW
COMMISSION_MANAGE

PARTNER_APPROVE
PARTNER_SUSPEND

REPORT_VIEW
REPORT_EXPORT

USER_MANAGE
AUDIT_VIEW
```

12. leads

```
leads
```

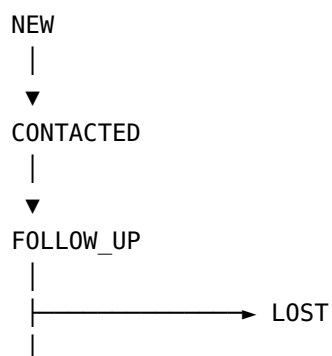
id	UUID PK
partner_id	UUID FK
assigned_to	UUID FK
name	VARCHAR
phone	VARCHAR
whatsapp	VARCHAR
email	VARCHAR
city	VARCHAR
course_id	UUID FK
source	VARCHAR
budget	DECIMAL
priority	ENUM
status	ENUM
score	INTEGER
follow_up_date	TIMESTAMP
notes	TEXT
closed_reason	TEXT
created_by	UUID FK
created_at	TIMESTAMP
updated_at	TIMESTAMP
archived_at	TIMESTAMP NULL

Possible priority:

LOW
MEDIUM
HIGH
HOT

13. Lead Status Model

Recommended state machine:



```
▼  
DEMO_BOOKED  
|  
▼  
DEMO_COMPLETED  
|  
▼  
FEE_DISCUSSION  
|  
▼  
ADMISSION_PENDING  
|  
▼  
ADMITTED
```

Alternative terminal state:

```
CLOSED / LOST
```

Invalid transitions must be rejected by the backend.

For example:

```
NEW → ADMITTED
```

should not be allowed directly.

14. lead_activities

Every important lead interaction should be represented as an activity.

```
lead_activities
```

```
id  
lead_id  
user_id  
type  
description  
metadata  
created_at
```

Activity types:

```
CREATED
ASSIGNED
STATUS_CHANGED
CALL
WHATSAPP
EMAIL
NOTE
DEMO_BOOKED
BROCHURE_SENT
FEE_DISCUSSION
ADMISSION_CREATED
```

This creates the lead timeline.

Example:

```
Lead Created
  ↓
Assigned to Rahul
  ↓
Call – Connected
  ↓
Brochure Sent
  ↓
Demo Booked
  ↓
Fee Discussed
  ↓
Admission Confirmed
```

15. follow_ups

```
follow_ups
_____
id
lead_id
assigned_to
due_at
priority
status
```

```
outcome
notes
completed_at
created_at
updated_at
```

Statuses:

```
PENDING
COMPLETED
OVERDUE
SNOOZED
CANCELLED
```

Outcomes:

```
CONNECTED
NO_RESPONSE
CALL_LATER
INTERESTED_LATER
DEMO_BOOKED
LOST
CONVERTED
```

16. admissions

```
admissions
_____
id
lead_id
partner_id
course_id
student_name
fee
payment_status
payment_mode
joining_date
remarks
verification_status
created_by
```

```
created_at
updated_at
```

Payment status:

```
UNPAID
PARTIAL
PAID
REFUNDED
CANCELLED
```

Verification:

```
PENDING
VERIFIED
REJECTED
```

17. payments

Although the MVP can keep payment fields inside Admission, a dedicated Payment entity is recommended if multiple payments are expected.

```
payments
└── id
    admission_id
    amount
    payment_mode
    transaction_reference
    payment_date
    status
    created_by
    created_at
```

This allows:

```
Admission Fee = ₹50,000

Payment 1 = ₹10,000
Payment 2 = ₹15,000
```

Payment 3 = ₹25,000

Total = ₹50,000

instead of overwriting a single payment field.

18. courses

`courses`

`id`
`title`
`description`
`duration`
`fee`
`syllabus_url`
`brochure_url`
`demo_url`
`benefits`
`faq`
`status`
`created_at`
`updated_at`

Course status:

`ACTIVE`
`INACTIVE`
`ARCHIVED`

19. Marketing Assets

`marketing_assets`

`id`
`course_id`
`title`
`type`
`file_url`

```
thumbnail_url  
language  
campaign_id  
status  
created_by  
created_at  
updated_at
```

Asset types:

```
POSTER  
BANNER  
VIDEO  
REEL  
CAPTION  
WHATSAPP_TEMPLATE  
BROCHURE  
TESTIMONIAL
```

20. Campaigns

```
campaigns  


---

id  
name  
description  
course_id  
status  
start_date  
end_date  
created_by  
created_at  
updated_at
```

Campaign analytics can later include:

```
sent  
delivered  
opened
```

clicked
converted

21. Commission Architecture

Commission rules should not be hard-coded into controllers.

Recommended structure:

```
CommissionRule
  |
  ▼
Admission
  |
  ▼
CommissionCalculator
  |
  ▼
CommissionRecord
```

Example:

```
Course A
Partner Type = Academy
Commission = 20%
```

Admission:

```
Fee = ₹50,000
Commission = ₹10,000
```

Commission record:

```
commission_records
├── id
│   partner_id
│   admission_id
│   rule_id
│   base_amount
```

```
commission_rate
commission_amount
status
payout_id
created_at
paid_at
```

Statuses:

```
PENDING
APPROVED
PAID
CANCELLED
```

22. Notification Architecture

Notifications should be treated as their own domain.

```
notifications
```

```
id
user_id
partner_id
type
title
message
reference_type
reference_id
read_at
created_at
```

Notification examples:

```
Follow-up due
Follow-up overdue
New lead assigned
Partner approved
Admission created
Payment received
```

Commission updated
Announcement published

23. Audit Logging

Sensitive actions must create audit events.

```
audit_logs
-----
id
user_id
partner_id
action
entity_type
entity_id
old_value
new_value
ip_address
user_agent
created_at
```

Examples:

```
PARTNER_APPROVED
PARTNER_SUSPENDED
ROLE_CHANGED
LEAD_TRANSFERRED
ADMISSION_UPDATED
COMMISSION_APPROVED
PAYOUT_APPROVED
USER_CREATED
USER_DEACTIVATED
```

Audit logs should be append-only.

Normal users must not be able to modify or delete them.

24. Authentication Architecture

Recommended:

```
Email + Password
    ↓
Password verification
    ↓
JWT access token
    +
Refresh token
    ↓
Authenticated API requests
```

Passwords:

```
Plain password
    ↓
Argon2id / bcrypt
    ↓
Password hash
```

Never store plaintext passwords.

25. JWT Payload

Keep JWT payload small.

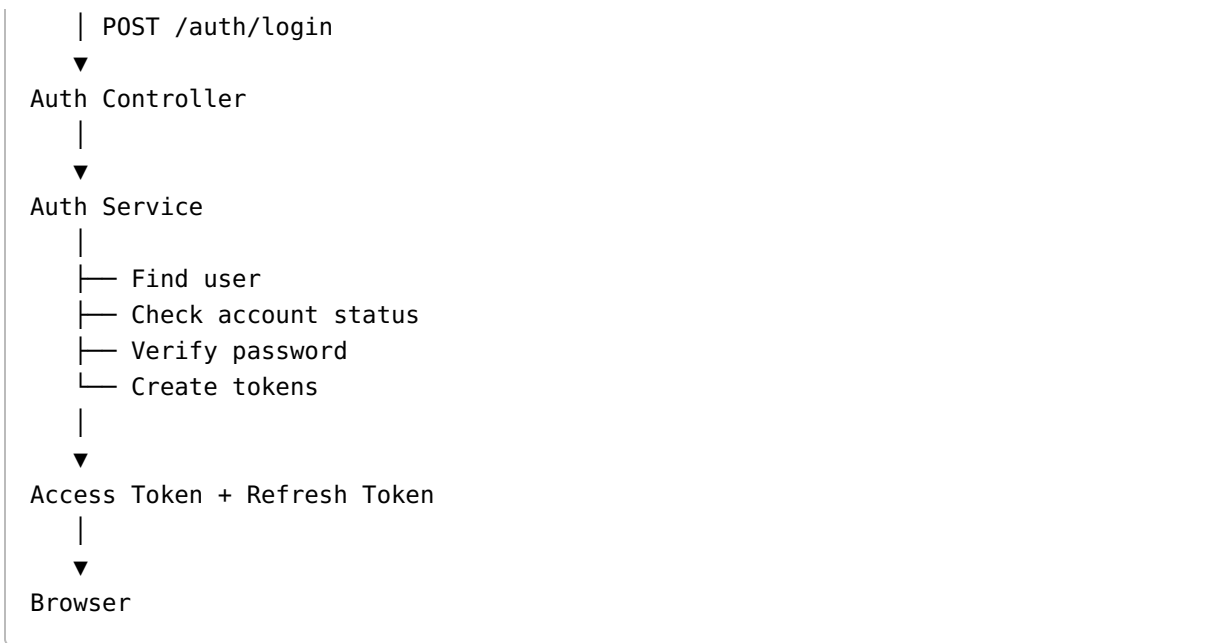
Example conceptual structure:

```
{
  "sub": "user-id",
  "role": "PARTNER_ADMIN",
  "partnerId": "partner-id",
  "sessionId": "session-id"
}
```

Do not place sensitive business data inside JWT.

26. Authentication Flow

```
Browser
|
```



Authenticated request:



27. Authorization Architecture

Authorization must exist at multiple levels.

```
Authentication
  ↓
Who are you?
  ↓
Authorization
  ↓
What role do you have?
  ↓
Permission
  ↓
What operation can you perform?
  ↓
Tenant Scope
  ↓
Which records can you access?
  ↓
Ownership / Assignment
  ↓
Which specific records can you modify?
```

Example:

```
Counselor
  ↓
LEAD_UPDATE permission
  ↓
Same partner?
  ↓
Lead assigned to counselor?
  ↓
Allow update
```

28. API Architecture

Base API:

```
/api/v1
```

Authentication:

```
POST /api/v1/auth/login
POST /api/v1/auth/refresh
POST /api/v1/auth/logout
POST /api/v1/auth/forgot-password
POST /api/v1/auth/reset-password
GET  /api/v1/auth/me
```

29. Partner APIs

```
GET    /api/v1/partners
POST   /api/v1/partners
GET    /api/v1/partners/:id
PATCH /api/v1/partners/:id
POST   /api/v1/partners/:id/approve
POST   /api/v1/partners/:id/reject
POST   /api/v1/partners/:id/suspend
```

Super Admin only for administrative partner operations.

30. User APIs

```
GET    /api/v1/users
POST   /api/v1/users
GET    /api/v1/users/:id
PATCH /api/v1/users/:id
PATCH /api/v1/users/:id/status
PATCH /api/v1/users/:id/role
```

31. Lead APIs

```
GET    /api/v1/leads
POST   /api/v1/leads
GET    /api/v1/leads/:id
PATCH /api/v1/leads/:id
DELETE /api/v1/leads/:id

POST   /api/v1/leads/:id/assign
POST   /api/v1/leads/:id/status
POST   /api/v1/leads/:id/archive

GET    /api/v1/leads/:id/activities
POST   /api/v1/leads/:id/activities

GET    /api/v1/leads/:id/follow-ups
POST   /api/v1/leads/:id/follow-ups
```

32. Admission APIs

```
GET    /api/v1/admissions
POST   /api/v1/admissions
GET    /api/v1/admissions/:id
PATCH /api/v1/admissions/:id

POST   /api/v1/admissions/:id/verify
POST   /api/v1/admissions/:id/cancel

GET    /api/v1/admissions/:id/payments
POST   /api/v1/admissions/:id/payments
```

33. Course APIs

```
GET    /api/v1/courses
POST   /api/v1/courses
GET    /api/v1/courses/:id
PATCH /api/v1/courses/:id
DELETE /api/v1/courses/:id
```

34. Commission APIs

```
GET    /api/v1/commissions
GET    /api/v1/commissions/:id

GET    /api/v1/commission-rules
POST   /api/v1/commission-rules
PATCH /api/v1/commission-rules/:id

POST   /api/v1/payouts
POST   /api/v1/payouts/:id/approve
POST   /api/v1/payouts/:id/process
```

35. Reporting APIs

```
GET /api/v1/reports/dashboard
GET /api/v1/reports/leads
GET /api/v1/reports/admissions
GET /api/v1/reports/revenue
GET /api/v1/reports/commission
GET /api/v1/reports/conversion
GET /api/v1/reports/partners
GET /api/v1/reports/courses
GET /api/v1/reports/sources
```

Common query parameters:

```
from
to
partnerId
courseId
status
source
city
assignedTo
page
limit
sort
```

36. Dashboard Architecture

Dashboard should not perform dozens of independent frontend queries.

Instead:

```
GET /api/v1/dashboard
```

or:

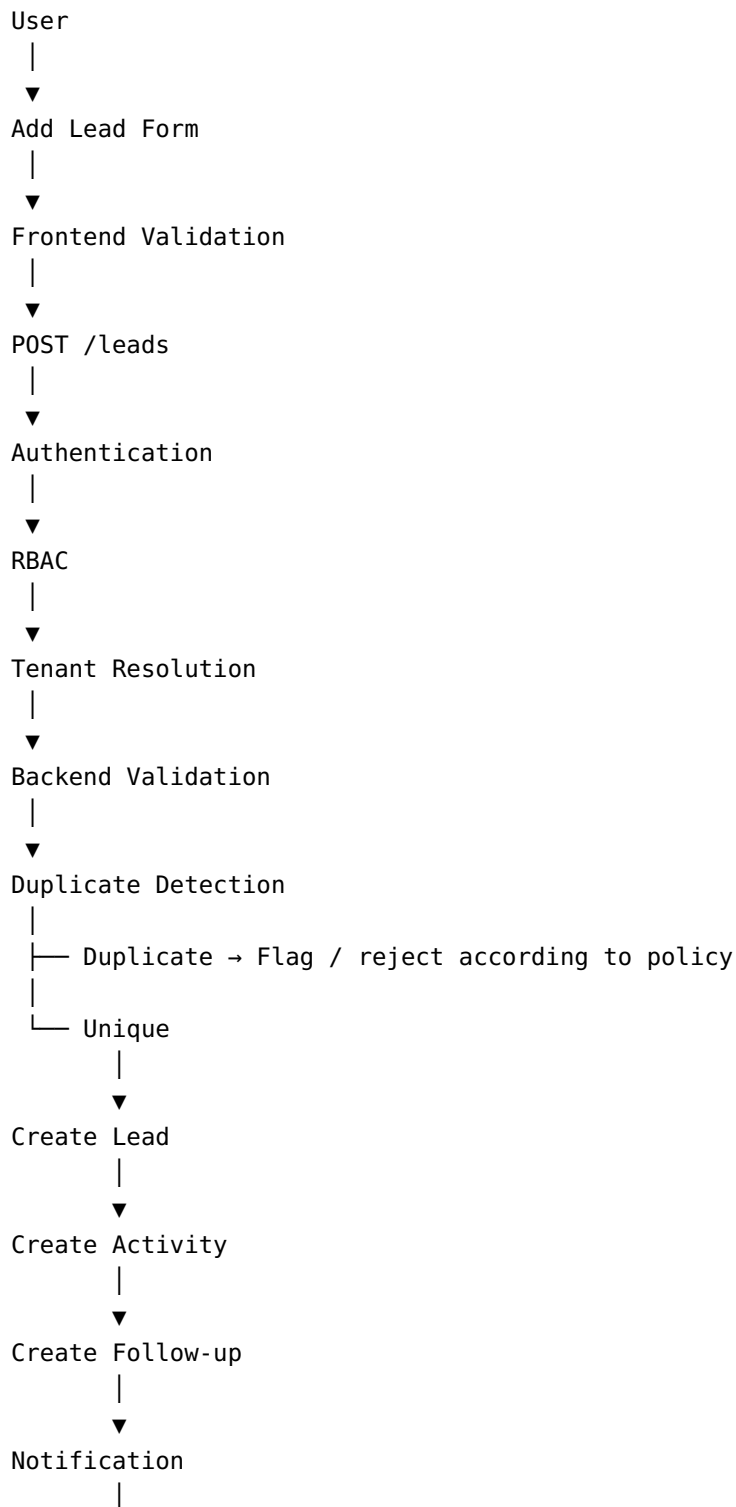
```
GET /api/v1/reports/dashboard
```

returns aggregated information.

Example:

```
{
  "leads": {
    "total": 1200,
    "hot": 145,
    "new": 82
  },
  "followUps": {
    "today": 24,
    "overdue": 8
  },
  "admissions": {
    "total": 145,
    "thisMonth": 21
  },
  "revenue": {
    "thisMonth": 1250000
  },
  "commission": {
    "pending": 175000,
    "paid": 425000
  }
}
```

37. Lead Creation Workflow



▼
Response

38. Duplicate Detection

Initial duplicate strategy:

```
phone exact match  
OR  
email exact match
```

Scoped primarily to the relevant tenant unless Super Admin is intentionally performing global detection.

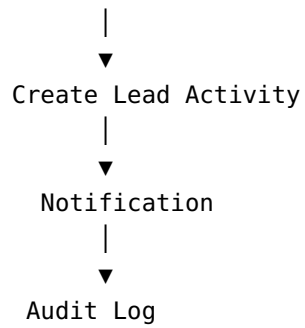
Potential future enhancement:

```
phone  
email  
name similarity  
city  
course  
historical records
```

AI/fuzzy duplicate detection belongs in a future phase.

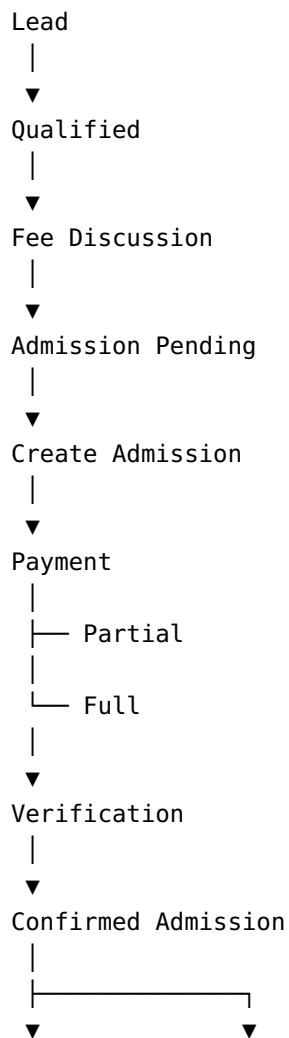
39. Lead Assignment Workflow

```
Super Admin / Partner Admin  
|  
▼  
Select Lead  
|  
▼  
Select Counselor  
|  
▼  
Validate Same Tenant  
|  
▼  
Update assigned_to
```



A counselor must never be assignable across unrelated partners.

40. Lead-to-Admission Workflow



Commission Reports
Calculation

Important invariant:

Lead status = ADMITTED

should only occur when:

Admission record exists

41. Admission Transaction

Admission creation should be atomic.

Conceptually:

```
BEGIN TRANSACTION

1. Validate lead
2. Validate course
3. Validate tenant
4. Verify admission doesn't already exist
5. Create admission
6. Update lead status
7. Create activity
8. Calculate commission
9. Create commission record
10. Create notification
11. Create audit log

COMMIT
```

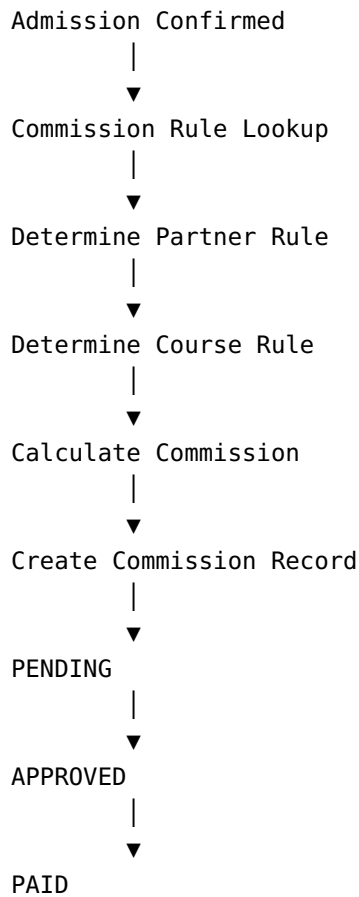
If a critical operation fails:

ROLLBACK

This prevents situations such as:

```
Lead = ADMITTED  
but  
Admission = missing
```

42. Commission Workflow



Commission calculation must be implemented in a dedicated service.

Example:

```
calculateCommission(admission)
```

not:

```
controller.calculateCommission()
```

43. Follow-up Reminder Architecture

For Phase 1:

```
Scheduled Worker
  |
  ▼
Find due follow-ups
  |
  ▼
Create notifications
  |
  ▼
Mark reminder generated
```

Future:

```
Worker
  |
  ├── In-app notification
  ├── Email
  ├── WhatsApp
  └── SMS
```

44. Notification Flow

```
Business Event
  |
  ▼
Event / Service
  |
  ▼
Notification Service
  |
  ├── Database notification
  |
```

```
├─ Email [future]
├─ WhatsApp [future]
└─ SMS [future]
```

The core CRM should not directly depend on WhatsApp.

Instead:

```
CRM
↓
Notification abstraction
↓
Provider
```

This makes future provider changes easier.

45. Reporting Architecture

Reporting should use optimized database queries.

Example:

```
Lead table
+
Admission table
+
Payment table
+
Partner table
+
Course table
↓
Aggregation Queries
↓
Report Service
↓
Dashboard / Export
```

Reports:

Lead Funnel
Conversion Rate
Revenue
Admission Count
Partner Performance
Course Performance
Source Performance
City Performance
Commission

46. Conversion Funnel

Example:

```
graph TD; A[1200 Leads] --> B[800 Contacted]; B --> C[500 Interested]; C --> D[300 Demo]; D --> E[180 Fee Discussion]; E --> F[120 Admissions];
```

1200 Leads
↓
800 Contacted
↓
500 Interested
↓
300 Demo
↓
180 Fee Discussion
↓
120 Admissions

Conversion:

$$\text{Admissions} / \text{Total Leads} \times 100$$

47. Export Architecture

For small datasets:

```
Request
↓
Generate CSV/XLSX/PDF
↓
Return download
```

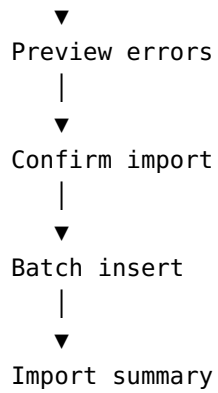
For large datasets:

```
Export Request
↓
Create Export Job
↓
Queue
↓
Worker
↓
Generate File
↓
Object Storage
↓
Notification
↓
Download
```

48. Import Architecture

Lead import:

```
CSV/XLSX
|
▼
Upload
|
▼
Validate headers
|
▼
Validate rows
|
▼
Duplicate detection
|
```



Example result:

```
1000 rows processed

Valid:      920
Duplicates: 55
Invalid:    25
Imported:   920
```

49. Frontend Architecture

Recommended:

```
Next.js
TypeScript
Tailwind CSS
React
TanStack Query
React Hook Form
Zod
```

Conceptual structure:

```
app/
|
├── (auth)/
|   ├── login/
|   └── forgot-password/
|
```

```
├─ dashboard/
├─ leads/
│   ├─ page
│   ├─ [id]/
│   └─ new/
├─ follow-ups/
├─ admissions/
├─ courses/
├─ marketing/
├─ commissions/
├─ reports/
├─ notifications/
├─ settings/
└─ admin/
    ├─ partners/
    ├─ users/
    └─ audit-logs/
```

50. Frontend Feature Organization

Avoid putting all logic into pages.

Preferred:

```
features/
├─ auth/
├─ leads/
├─ followups/
├─ admissions/
├─ courses/
├─ commissions/
├─ reports/
├─ notifications/
└─ partners/
```

Each feature can contain:

```
components/
hooks/
```

```
api/  
schemas/  
types/  
utils/
```

51. Backend Architecture

Recommended structure:

```
src/  
|  
├─ config/  
|  
├─ middleware/  
|  
├─ modules/  
|   |  
|   ├─ auth/  
|   ├─ users/  
|   ├─ partners/  
|   ├─ leads/  
|   ├─ followups/  
|   ├─ admissions/  
|   ├─ payments/  
|   ├─ courses/  
|   ├─ marketing/  
|   ├─ commissions/  
|   ├─ notifications/  
|   ├─ reports/  
|   ├─ audit/  
|   └─ imports/  
|  
├─ database/  
├─ jobs/  
├─ shared/  
|   ├─ errors/  
|   ├─ validation/  
|   ├─ pagination/  
|   └─ utilities/  
|  
├─ app.ts  
└─ server.ts
```

52. Backend Module Pattern

Example:

```
modules/leads/  
  
lead.controller.ts  
lead.service.ts  
lead.repository.ts  
lead.routes.ts  
lead.schema.ts  
lead.types.ts
```

Responsibilities:

Controller

HTTP concerns only.

```
Request  
Response  
Status codes
```

Service

Business logic.

```
Duplicate detection  
Status transitions  
Assignment rules  
Tenant validation
```

Repository

Database interaction.

```
findLead()  
createLead()  
updateLead()
```

Schema

Input validation.

```
phone
email
course
priority
status
```

53. API Error Architecture

Consistent error format:

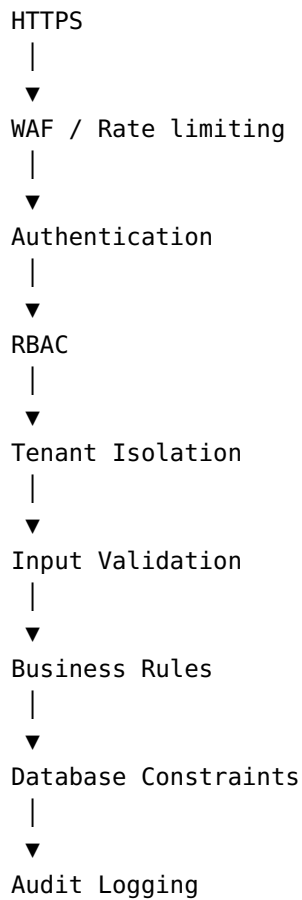
```
{
  "timestamp": "2026-09-10T10:30:00Z",
  "status": 400,
  "message": "Validation failed",
  "fieldErrors": {
    "phone": "Invalid phone number"
  },
  "path": "/api/v1/leads"
}
```

Common status codes:

```
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
409 Conflict
422 Unprocessable Entity
429 Too Many Requests
500 Internal Server Error
```

54. Security Architecture

Security layers:



55. Security Requirements

Implement:

- secure password hashing
- JWT expiration
- refresh-token rotation
- HTTPS
- input validation
- output encoding
- SQL injection protection through parameterized ORM/query APIs
- XSS protection
- CSRF protection where applicable
- rate limiting
- login throttling
- secure cookies where applicable
- environment-based secrets
- audit logging

- least-privilege database credentials
- controlled exports
- account suspension
- session invalidation

Never trust:

```
partnerId
role
userId
permissions
```

supplied by the frontend.

56. Database Constraints

Application validation is not enough.

Important database constraints:

```
users.email UNIQUE

partner membership valid

foreign keys enforced

required fields NOT NULL

commission amounts non-negative

payment amounts non-negative
```

For tenant-sensitive uniqueness, consider composite indexes such as:

```
(partner_id, phone)
(partner_id, email)
```

depending on the finalized duplicate policy.

57. Database Indexing

Important indexes:

```
users.email

leads.partner_id
leads.assigned_to
leads.status
leads.phone
leads.email
leads.follow_up_date
leads.course_id
leads.created_at

follow_ups.lead_id
follow_ups.assigned_to
follow_ups.due_at
follow_ups.status

admissions.partner_id
admissions.course_id
admissions.payment_status
admissions.created_at

payments.admission_id

commission_records.partner_id
commission_records.status

audit_logs.user_id
audit_logs.partner_id
audit_logs.created_at
```

Do not blindly index every column.

Indexes should follow actual query patterns.

58. Caching

Redis is optional for the earliest MVP but useful as the platform grows.

Good cache candidates:

```
Course catalog
Course FAQs
System settings
Permission metadata
Dashboard aggregates
```

Avoid caching highly volatile transactional data unnecessarily.

Example:

```
Lead status
Payment state
Commission state
```

should primarily come from PostgreSQL.

59. Background Jobs

Use a worker/queue architecture for operations that don't need to block an HTTP request.

Candidates:

```
Follow-up reminders
Email notifications
WhatsApp messages
Bulk lead imports
Large report generation
Export generation
Marketing campaign processing
Analytics aggregation
```

Conceptually:

```
API
|
▼
Queue
|
▼
Worker
|
```



Task

60. Observability

The system should provide:

```
Application logs
Error logs
Audit logs
Request IDs
Performance metrics
Health checks
```

Example:

```
GET /api/v1/leads/123
requestId = req_abc123
userId = usr_456
partnerId = partner_789
duration = 142ms
status = 200
```

Sensitive information such as passwords and tokens must never be logged.

61. Health Checks

Recommended endpoints:

```
GET /health
GET /health/live
GET /health/ready
```

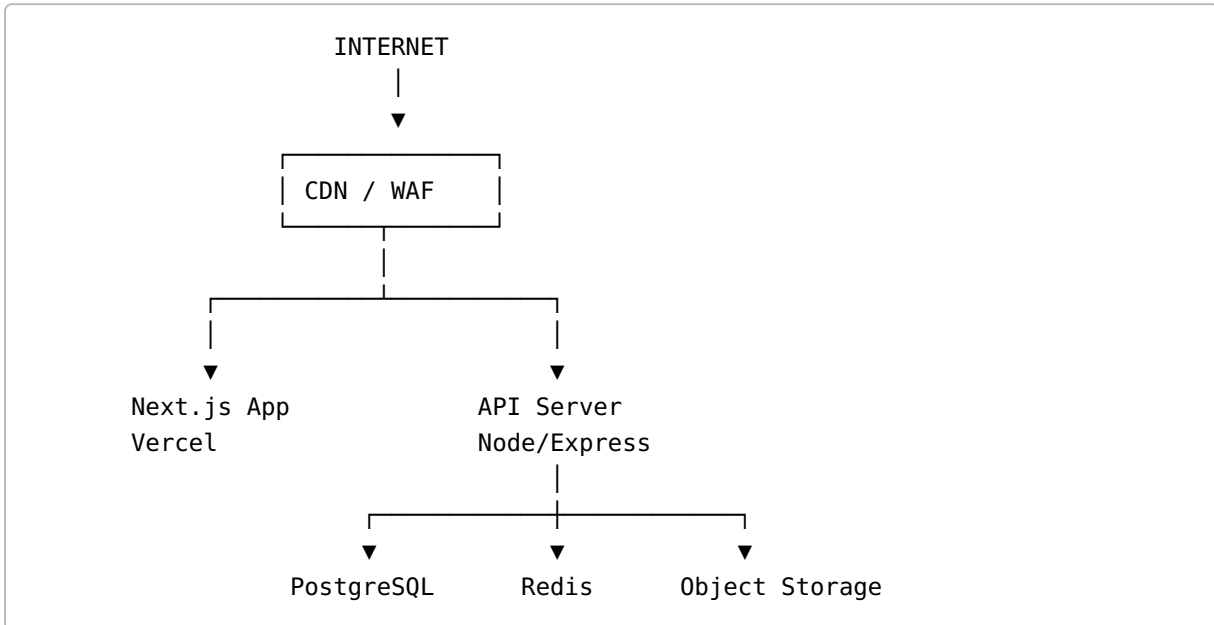
Example:

```
/liveness
  API process running?
```

```
/readiness
  API + database + required dependencies ready?
```

62. Deployment Architecture

Recommended production arrangement:



Possible providers:

Frontend:

Vercel

Backend:

AWS / Render / Railway / VPS

Database:

Managed PostgreSQL

Storage:

S3-compatible object storage

The exact provider can be selected after deployment requirements and budget are finalized.

63. Environment Separation

Maintain:

```
development
staging
production
```

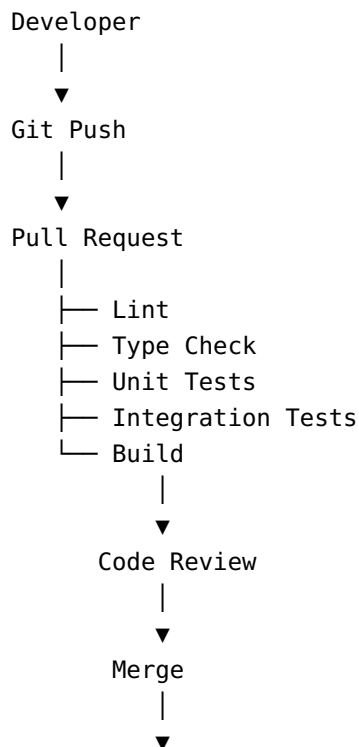
Never use production credentials locally.

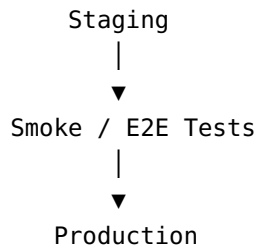
Example:

```
.env.local
.env.staging
.env.production
```

Secrets should preferably be stored in the hosting provider's secret/environment management system.

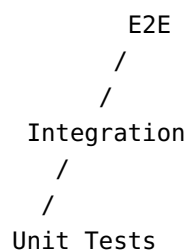
64. CI/CD Pipeline





65. Testing Strategy

Testing pyramid:



Unit Tests

Test:

- Commission calculation
- Lead status transitions
- Duplicate detection
- Permission checks
- Validation

Integration Tests

Test:

- API + PostgreSQL
- Authentication
- Tenant isolation
- Lead creation

Admission conversion
Commission creation

E2E Tests

Test critical workflows:

```
Login
↓
Partner Dashboard
↓
Create Lead
↓
Assign Lead
↓
Schedule Follow-up
↓
Convert Admission
↓
Payment
↓
Commission
↓
Report
```

66. Critical Security Test

One of the most important tests:

```
Partner A logs in
↓
Attempts to request Partner B lead
↓
API checks tenant
↓
403 Forbidden
```

The system must never depend solely on frontend route hiding.

67. Role Matrix

Feature	Super Admin	Partner Admin	Counselor	Support
Global Dashboard	✓	—	—	—
Own Dashboard	✓	✓	✓	✓
Partner Management	✓	—	—	Limited
Lead View	All	Own	Assigned	Limited
Lead Create	✓	✓	✓	—
Lead Assignment	✓	✓	Limited	—
Follow-ups	✓	✓	Assigned	—
Admissions	✓	✓	Limited	—
Courses	✓	View	View	View
Marketing	✓	✓	✓	Limited
Commission	✓	Own	—	—
Reports	All	Own	Limited	Limited
Export	✓	Permission-based		—
User Management	✓	Own team		—
Audit Logs	✓	—	—	—
System Settings	✓	—	—	—

The final permission matrix should be implemented through permissions rather than hard-coding role names throughout the application.

68. Frontend Navigation

Super Admin

Dashboard

Partners

└─ All Partners

└─ Pending Approval

└─ Partner Performance

Leads
├─ All Leads
├─ Unassigned
└─ Duplicate Leads

Admissions

Courses

Marketing

Commissions

Reports

Notifications

Users & Roles

Audit Logs

Settings

Partner Admin

Dashboard

Leads
├─ All Leads
├─ Add Lead
└─ Follow-ups

Admissions

Courses

Marketing

Commissions

Reports

Team

Notifications

Settings

Counselor

Dashboard

My Leads

Follow-ups

Admissions

Courses

Marketing

Notifications

Profile

69. Dashboard Design

Super Admin dashboard:

Total Partners | Leads | Admissions | Revenue

Lead Funnel

Admission Trend

Partner Rankings

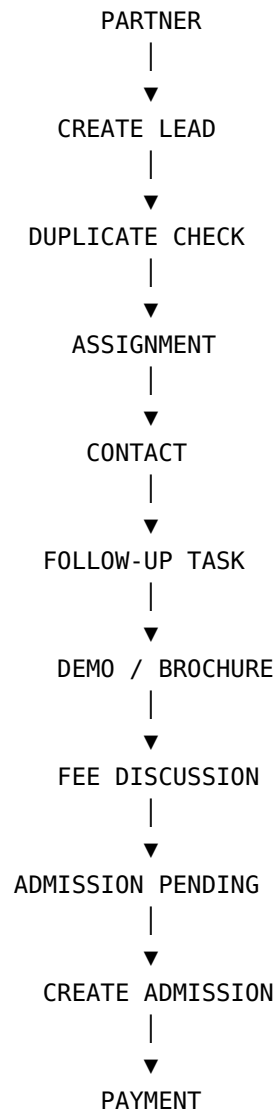
Course Performance

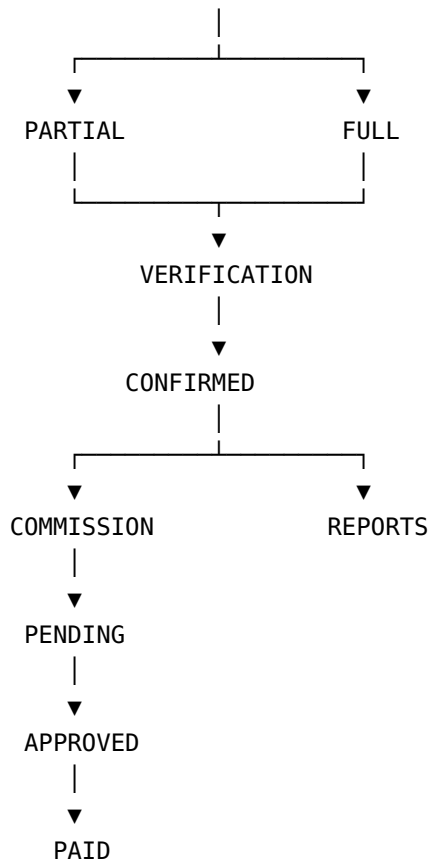
Urgent Follow-ups

Partner dashboard:

Total Leads
Hot Leads
Today's Follow-ups
Admissions
Conversion Rate
Pending Commission
Paid Commission

70. Data Flow — Complete Admission Lifecycle





71. System Boundaries

Inside Phase 1

Authentication
RBAC
Partners
Users
Leads
Follow-ups
Admissions
Courses
Basic Reports
Dashboard
Audit

Phase 2

Marketing
Commission
Notifications
Import/Export
Advanced Reports

Phase 3

WhatsApp
Email automation
Payment gateway
Automated reminders
Campaigns

Phase 4

AI Lead Scoring
Prediction
Next-best action
Smart recommendations

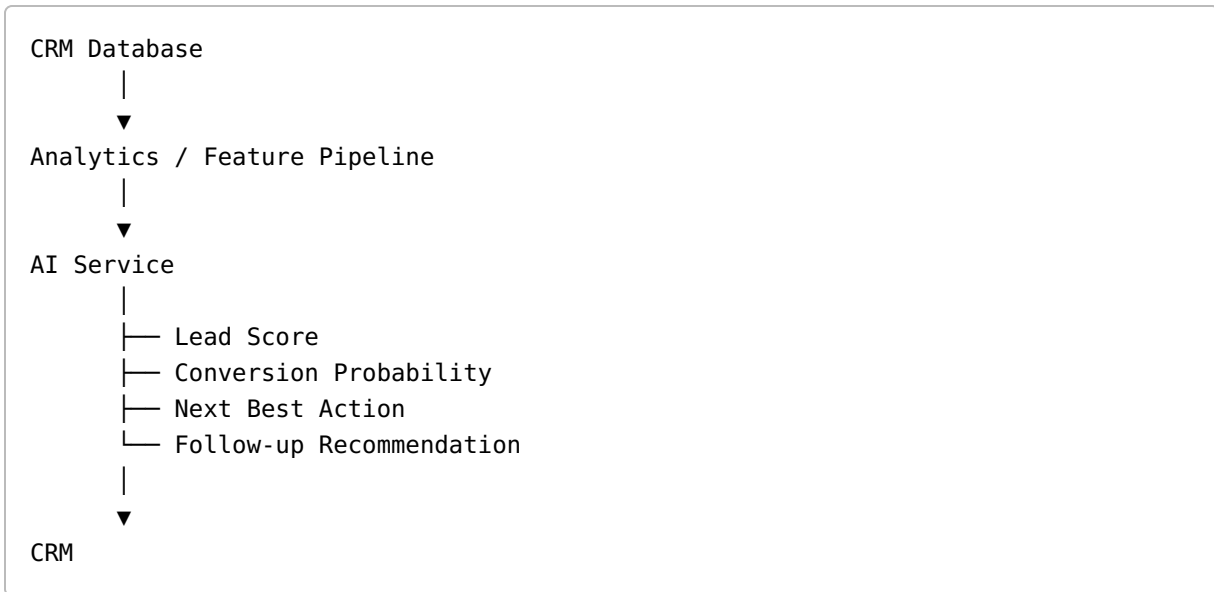
Phase 5

LMS
Quizzes
Certificates
Mobile application
Franchise management

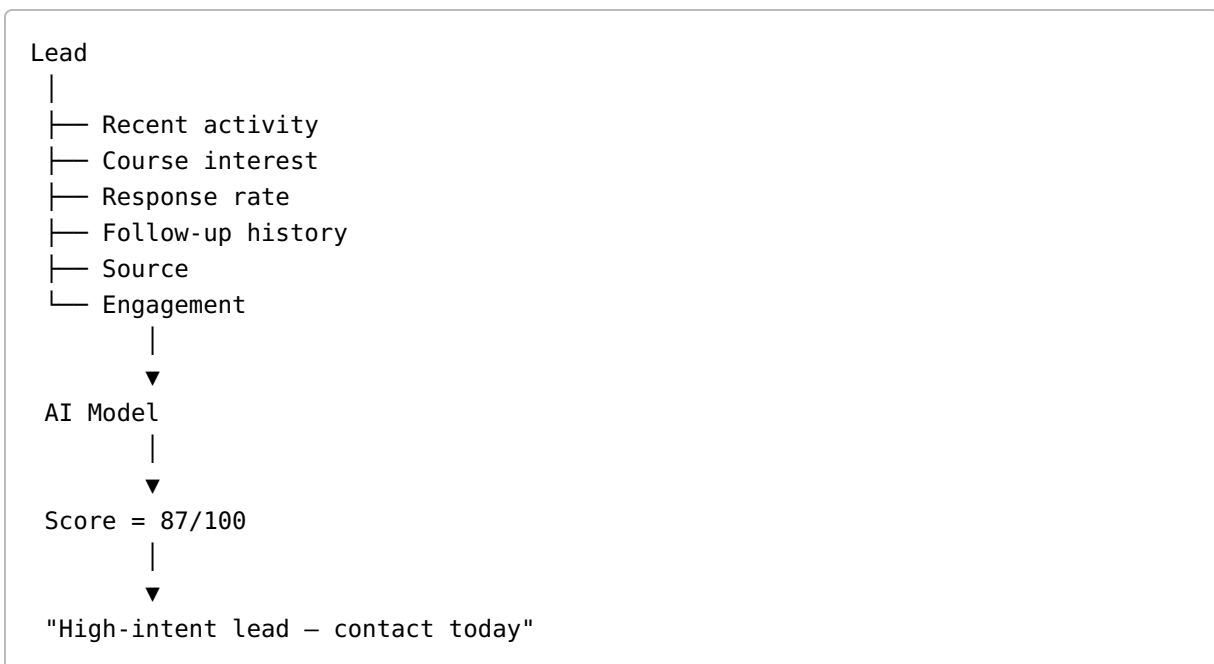
72. Future AI Architecture

AI should not be embedded directly into core transactional services.

Recommended future architecture:



Example:

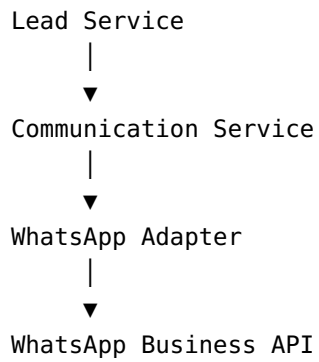


AI must augment the CRM rather than become the source of truth for transactional state.

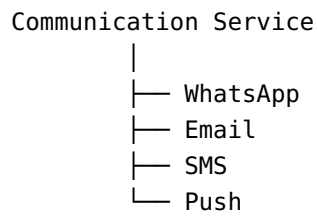
73. Future WhatsApp Architecture

Do not make the Lead service directly dependent on WhatsApp.

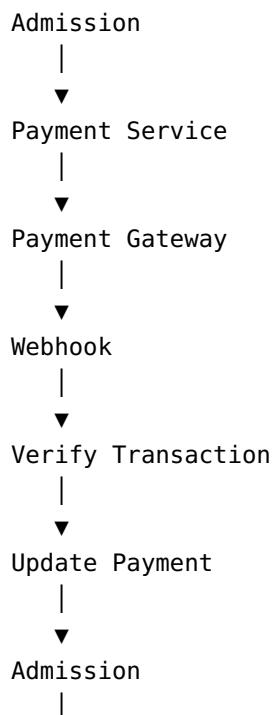
Use:



Later:



74. Future Payment Architecture



▼
Commission

Payment gateway callbacks must be independently verified rather than trusting the browser.

75. Key Architectural Invariants

The following rules must always remain true:

Invariant 1

Partner user cannot access another partner's records.

Invariant 2

Lead marked ADMITTED
→ admission record must exist.

Invariant 3

Admission commission
→ must reference a valid commission rule/calculation.

Invariant 4

Audit records are append-only.

Invariant 5

Unauthorized frontend actions must also fail at the API.

Invariant 6

A suspended user cannot authenticate into the system.

Invariant 7

Counselor can operate only on permitted assigned records.

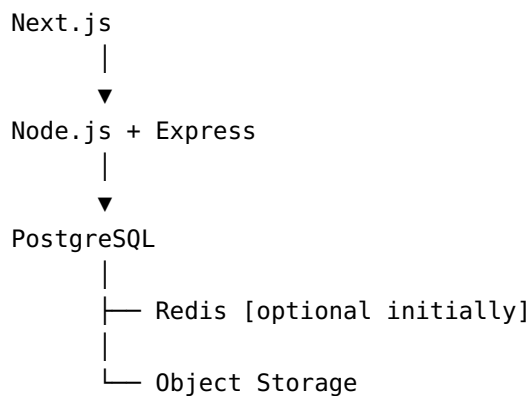
Invariant 8

Financial records should never be silently overwritten.

Use payment records/history instead of destructive updates.

76. Recommended MVP Architecture

For the first production version:



Core modules:

Auth
Users
Partners
Leads
Follow-ups
Admissions
Courses
Dashboard
Reports
Audit

Do not build these prematurely:

- Microservices
- Kubernetes
- AI infrastructure
- Event streaming platform
- Complex workflow engine
- Real-time collaboration
- Mobile app
- Payment gateway
- WhatsApp automation

Build the transactional core first.

77. Suggested Development Phases

Phase 0 — Foundation

- Repository
- Environment configuration
- Database
- Migrations
- API framework
- Frontend framework
- CI/CD
- Error handling
- Logging

Phase 1 — Identity

- Authentication
- Users
- Roles
- Permissions
- Partner tenancy
- Session management

Phase 2 — CRM Core

- Partners
- Leads
- Assignments

Lead activities
Follow-ups
Lead timeline

Phase 3 — Admissions

Admission
Payment tracking
Course integration
Admission verification

Phase 4 — Business Intelligence

Dashboard
Reports
Conversion funnel
Partner performance
Course performance

Phase 5 — Commercial

Commission rules
Commission records
Payouts

Phase 6 — Operations

Marketing
Notifications
Import
Export

Phase 7 — Integrations

WhatsApp
Email
Payments

Phase 8 — Intelligence

AI scoring
Predictions
Recommendations
Automation

78. Recommended Repository Structure

A monorepo is appropriate:

```
whitedavid23_academy/
├── apps/
│   ├── web/
│   │   └── Next.js application
│   └── api/
│       └── Express backend
├── packages/
│   ├── types/
│   ├── validation/
│   ├── ui/
│   └── config/
├── database/
│   ├── migrations/
│   └── seeds/
├── docs/
│   ├── architecture/
│   ├── api/
│   ├── database/
│   └── workflows/
├── .github/
│   └── workflows/
└── docker/
```

```
├─ README.md
└─ package.json
```

A simpler two-repository architecture is also valid:

```
whitedavid23-web
whitedavid23-api
```

The choice can depend on team size and deployment workflow.

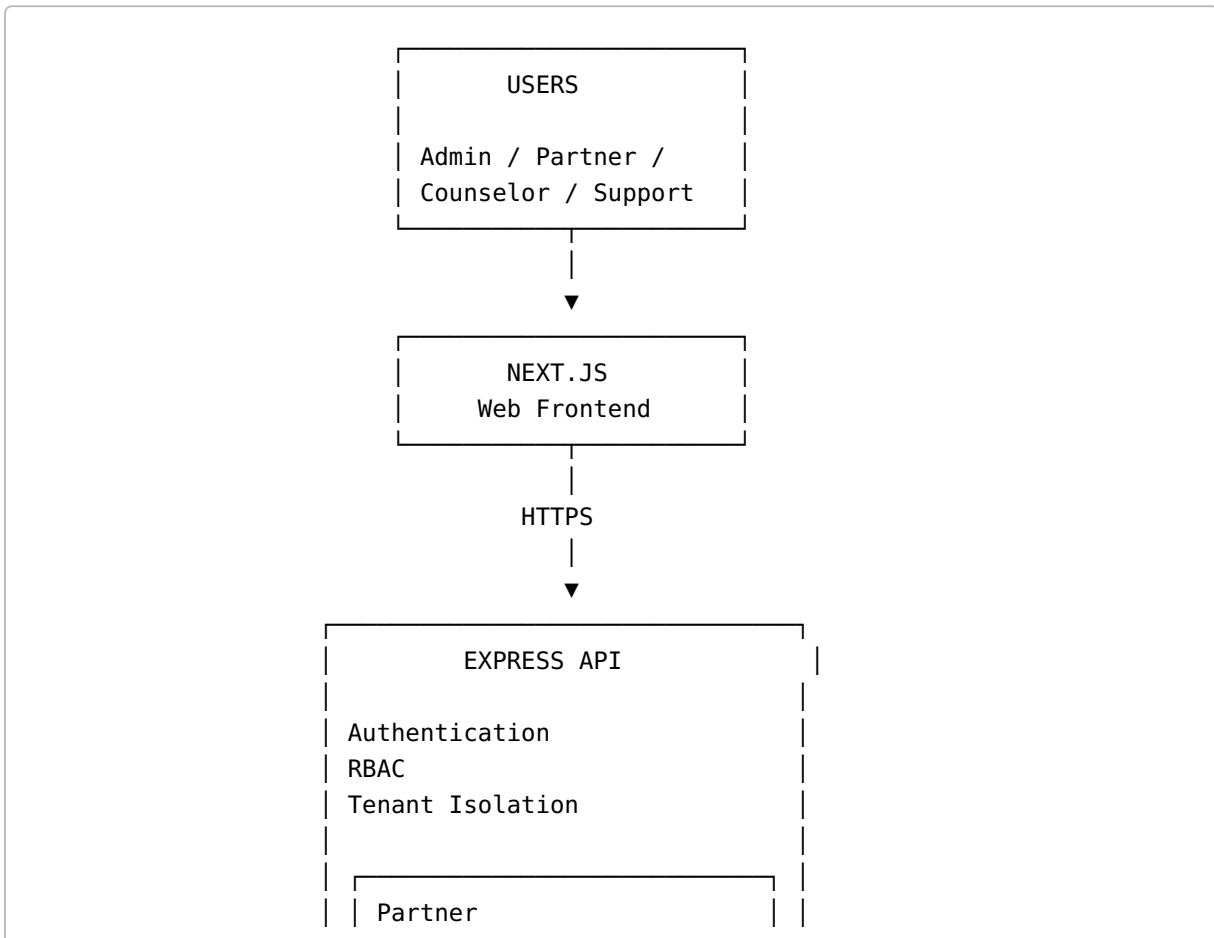
79. Architecture Decision Summary

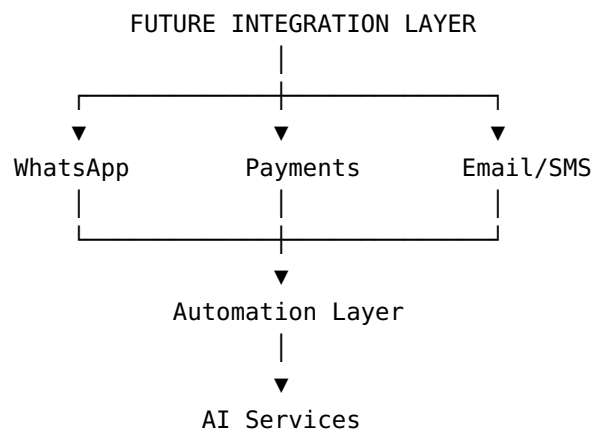
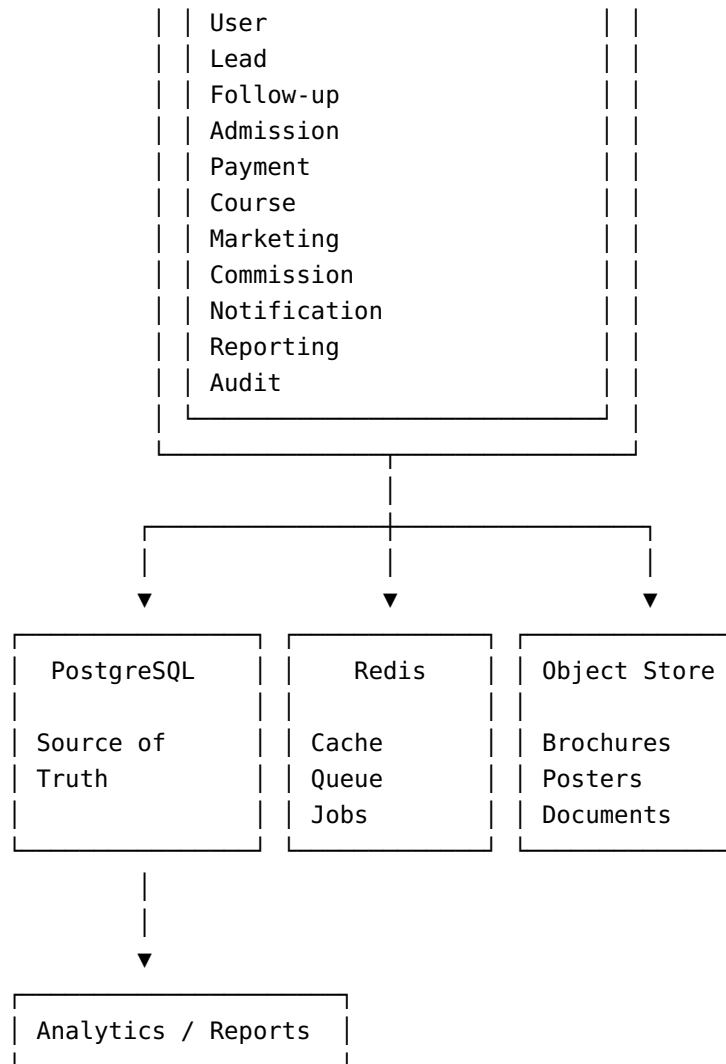
Area	Decision
Architecture	Modular Monolith
Frontend	Next.js + TypeScript
UI	Tailwind CSS
Backend	Node.js + Express
Database	PostgreSQL
ORM	Prisma / Drizzle / equivalent
Authentication	JWT + Refresh Token
Authorization	RBAC + Permission checks
Tenancy	Shared DB + <code>partner_id</code> isolation
Cache	Redis when required
Queue	Redis-backed worker/queue
File Storage	S3-compatible object storage
API	REST <code>/api/v1</code>
Validation	Zod / equivalent
Reporting	SQL aggregation
Logging	Structured application logs
Audit	Append-only audit records
Frontend Hosting	Vercel or equivalent

Area	Decision
Backend Hosting	AWS / Render / equivalent
Database	Managed PostgreSQL
CI/CD	GitHub Actions
Testing	Unit + Integration + E2E
AI	Separate future service
WhatsApp	Adapter/integration layer
Payments	Dedicated payment service
Deployment Style	Container/cloud compatible

80. Final Architecture

The resulting system can be visualized as:





81. Final Engineering Recommendation

The most important architectural principle for `whitedavid23_academy` is:

Build the CRM transactionally and modularly first; add automation and AI around the stable core later.

The first version should therefore optimize for:

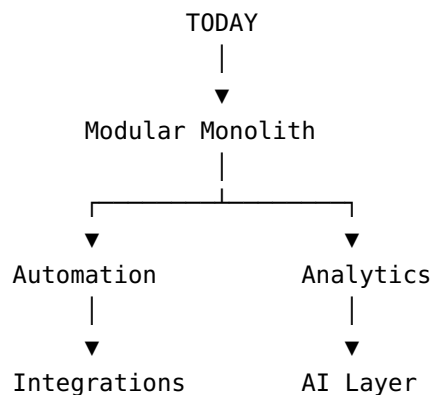
Correctness
↓
Security
↓
Tenant Isolation
↓
Maintainability
↓
Testability
↓
Performance
↓
Scalability

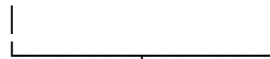
rather than prematurely optimizing for microservices or AI.

The **PostgreSQL database should remain the authoritative source of truth** for partners, users, leads, follow-ups, admissions, payments, commissions, and audit history.

The application should expose those capabilities through a versioned REST API, while the frontend remains a consumer of that API rather than containing business rules.

The architecture should consequently be capable of evolving:





FUTURE SCALE



Extract services only when
actual scale/ownership/
operational requirements
justify doing so.

This gives WhiteDavid23 a **production-oriented foundation without over-engineering the MVP.**